Inversion of Control (IoC) & Spring Container

Before understanding **Inversion of Control**, we must first understand **control itself**.

1️⃣ What Do We Mean by "Control"?

In traditional Java applications:

- Classes decide **what objects to create**
- Classes decide **when to create them**
- Classes manage **lifecycle and cleanup**

This may feel normal — but it is the **root cause of tight coupling**.

2️⃣ The Core Misunderstanding (Very Common)

Many developers think:

"Tight coupling exists only when I have multiple implementations
(Airtel, Jio, Idea).
If I have only one class, why worry?"

This sounds logical — but it's incomplete.

3️⃣ What Tight Coupling Actually Means

Tight coupling is NOT about number of classes.
It is about **who controls object creation**.

Tight Coupling
→ *This class decides what concrete dependency it will use.*

Loose Coupling
→ *This class does NOT decide. Someone else provides the dependency.*

Even **one single class** can be tightly coupled.

4️⃣ Single-Class Example (Very Important)

Case 1: Without DI (Tight Coupling)

```
class Mobile {
    private AirtelSim sim = new AirtelSim();
}
```

You might say:

“There’s only Airtel. What’s the issue?”

Hidden Problems:

1. Mobile is tied to AirtelSim
2. Cannot test Mobile independently
3. Cannot replace AirtelSim without code change
4. Mobile controls creation (violates SRP)
5. Any future change requires recompilation & redeployment

This is **tight coupling**, even with one class.

5️⃣ Why This Becomes Dangerous in Real Projects

Today:

- Only AirtelSim

Tomorrow:

- Needs database
- Needs API calls
- Needs caching
- Needs mocks for testing

Now every change forces:

- Code modification
- Rebuild
- Retest

- Redeploy

That's **fragile architecture**.

💉 Same Scenario with Dependency Injection (Loose Coupling)

```
interface Sim {
    void call();
}

class Mobile {
    private Sim sim;

    Mobile(Sim sim) {
        this.sim = sim;
    }
}
```

Now:

- Mobile depends on **abstraction**
- Creation happens **outside**
- Mobile has **zero knowledge** of concrete class

Even with one implementation today, you gain:

- Testability
- Replaceability
- Clean architecture
- SRP compliance
- Future safety

7️⃣ Real-Life Analogy

Tight Coupling (Even with One Option)

- SIM is soldered inside phone
- Cannot replace
- Cannot test without it
- Bad design

Loose Coupling (DI)

- Phone has SIM slot
- Even if only Airtel exists today
- Design is still correct
- Phone is future-proof

8️⃣ Why Spring Enforces DI Even with One Class

Spring is **not solving today's problem**.
Spring is solving **tomorrow's change**.

Spring assumes:

- Requirements will change
- Testing will be needed
- Implementations will grow
- Production issues will happen

So Spring enforces:

- Constructor injection
- Interface-driven design
- External wiring

Not for convenience — **for survivability**.

9️⃣ Interview-Grade One-Liner

“Tight coupling is not about how many implementations exist.
It’s about who controls object creation.
Dependency Injection removes that control from the class.”

🔁 Transition to Inversion of Control (IoC)

So far we said:

- Objects should be created **outside**
- Dependencies should be **injected**

Now the real question:

👉 **Who does this creation and wiring?**

0️⃣ Inversion of Control (IoC)

Inversion of Control means:

The responsibility of creating, wiring, and managing objects
is moved from the application to an external framework.

In Spring:

- That external framework is **Spring itself**

DI is the **mechanism**
IoC is the **principle**

1️⃣ Spring Container — The Heart of IoC

The **Spring Container**:

- Creates beans
- Injects dependencies
- Manages lifecycle
- Destroys beans

You give **classes**.
Spring gives **fully wired objects**.

2️⃣ Configuration Metadata — How Spring Knows What to Do

Spring needs instructions.

These instructions are called **Configuration Metadata**.

You can provide them via:

1. XML configuration
2. Annotations
3. Java-based configuration

This metadata tells Spring:

- What to create
- How to configure
- How to assemble dependencies

3️⃣ Types of Spring Containers

BeanFactory

- Lightweight
- Lazy initialization
- Suitable for simple apps

ApplicationContext

- Enterprise-ready
- Eager initialization
- Event handling, AOP, i18n

➞ Used in real-world applications.

4️⃣ DI in Layered Architecture (Enterprise View)

In a typical backend system:

- Controller → depends on Service
- Service → depends on Repository
- Repository → depends on DataSource

Spring injects dependencies **across all layers**, ensuring:

- Loose coupling
- Testability
- Maintainability

Final Takeaway

- DI removes **object creation**
- IoC removes **ownership**
- Spring Container manages **everything in between**